

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
TAKE HOME TASK
MODUL 11
EXCEPTION HANDLING



Disusun Oleh:

Ubaidulloh Zulkarnian (NIM: 105224024)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Laporan ini disusun untuk mendokumentasikan hasil pengerjaan tugas Take Home Task (THT) Modul 11 pada praktikum Pemrograman Berorientasi Objek. Fokus utama praktikum modul ini adalah implementasi praktis dari konsep Exception Handling (Penanganan Eksepsi) pada bahasa pemrograman Java. Exception handling merupakan mekanisme penting untuk mendeteksi dan menangani kesalahan runtime (runtime errors) agar aplikasi tidak mengalami crash secara tiba-tiba. Pada tugas ini, konsep tersebut diterapkan pada Sistem Reservasi Tiket Kereta Api (Java Express). Program ini memproses pemesanan tiket dengan melakukan validasi NIK penumpang, pencarian rute kereta berdasarkan kode, serta pengelolaan kapasitas kursi. Penerapan exception handling dilakukan dengan menggunakan blok try-catch-finally, pelemparan eksepsi (throw/throws), serta pendefinisian custom exceptions (baik checked exception maupun unchecked exception). Hal ini bertujuan untuk membuat sistem yang tangguh (robust), aman, dan mampu memberikan umpan balik (feedback) yang jelas kepada pengguna saat terjadi kesalahan.

II. Variabel

No	Nama Variabel	Tipe Data	Letak Kelas	Fungsi
1	kode	String	Kereta	Menyimpan kode unik kereta (misalnya: "K01").
2	nama	String	Kereta	Menyimpan nama kereta api (misalnya: "Argo Bromo").
3	rute	String	Kereta	Menyimpan rute perjalanan kereta api (misalnya: "JKT - SBY").
4	sisaKursi	int	Kereta	Menyimpan jumlah kursi yang masih tersedia.
5	daftarKereta	List<Kereta>	Reservasi	Menyimpan daftar seluruh kereta yang tersedia dalam sistem.
6	namaKereta	String	TiketHabisException	Menyimpan nama kereta yang tiketnya habis saat eksepsi dipicu.
7	sisaKursi	int	TiketHabisException	Menyimpan jumlah kursi tersisa pada kereta saat eksepsi dipicu.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Letak Kelas	Fungsi
1	Kereta(String, String, String, int)	Constructor	Kereta	Menginisialisasi objek Kereta dengan kode, nama, rute, dan kapasitas.
2	getKode()	Method (Functional)	Kereta	Mengembalikan kode unik kereta.
3	getNama()	Method (Functional)	Kereta	Mengembalikan nama kereta.
4	getRute()	Method (Functional)	Kereta	Mengembalikan rute perjalanan kereta.
5	getSisaKursi()	Method (Functional)	Kereta	Mengembalikan jumlah sisa kursi.
6	kurangiKursi(int)	Method (Procedural)	Kereta	Mengurangi sisa kursi berdasarkan jumlah tiket yang dipesan.
7	toString()	Method (Functional)	Kereta	Mengembalikan representasi string info kereta dengan format 'kode nama rute sisaKursi'.
8	Reservasi()	Constructor	Reservasi	Menginisialisasi list kereta dan memasukkan data mock (Argo Bromo dan Parahyangan).
9	getDaftarKereta()	Method (Functional)	Reservasi	Mengembalikan daftar seluruh kereta.
10	pesanTiket(String, String, String, int)	Method (Procedural)	Reservasi	Memproses pemesanan tiket dengan validasi NIK (harus 16 angka), validasi rute (kode kereta harus ada), dan ketersediaan kursi. Dapat melempar RuteTidakDitemukanException atau TiketHabisException.

11	DataPenumpangTidakValidException(String)	Constructor	DataPenumpangTidakValidException	Menginisialisasi objek unchecked exception untuk data penumpang.
12	RuteTidakDitemukanException(String)	Constructor	RuteTidakDitemukanException	Menginisialisasi objek checked exception untuk rute tidak ditemukan.
13	TiketHabisException(String, String, int)	Constructor	TiketHabisException	Menginisialisasi objek checked exception untuk sisa tiket habis/tidak cukup.
14	getNamaKereta()	Method (Functional)	TiketHabisException	Mengembalikan nama kereta dari exception.
15	getSisaKursi()	Method (Functional)	TiketHabisException	Mengembalikan sisa kursi dari exception.
16	main(String[])	Method (Procedural)	App	Titik masuk aplikasi (main entry point) yang mengelola menu interaktif konsol.
17	tampilkanJadwal(Reservasi)	Method (Procedural)	App	Menampilkan daftar kereta dan sisa kursi ke layar.
18	prosesBoking(Scanner, Reservasi)	Method (Procedural)	App	Mengumpulkan input pemesanan dari pengguna, memanggil pesanTiket, dan menangani berbagai exception.

IV. Dokumentasi dan Pembahasan Code

1. Dokumentasi Source Code

- App.java: Menyediakan driver utama program, menampilkan menu interaktif konsol, membaca input pengguna, serta memanggil Reservasi.pesanTiket dengan penanganan exception menggunakan block try-catch.

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class App {
5     public static void main(String[] args) {
6         Scanner scanner = new Scanner(System.in);
7         Reservasi reservasi = new Reservasi();
8
9         System.out.println("=====");
10        System.out.println(" Selamat Datang di JAVA EXPRESS");
11        System.out.println(" Sistem Reservasi Tiket Kereta Api");
12        System.out.println("=====");
13
14        boolean running = true;
15
16        try {
17            while (running) {
18                System.out.println("\n-- MENU UTAMA --");
19                System.out.println("1. Lihat Jadwal Kereta");
20                System.out.println("2. Pesan Tiket");
21                System.out.println("3. Keluar");
22                System.out.print("Pilih menu: ");
23
24                int pilihan;
25                try {
26                    pilihan = scanner.nextInt();
27                    scanner.nextLine(); // bersihkan buffer
28                } catch (InputMismatchException e) {
29                    System.out.println("[ERROR] Input tidak valid. Masukkan angka 1-3.");
30                    scanner.nextLine();
31                    continue;
32                }
33
34                switch (pilihan) {
35                    case 1:
36                        tampilkanJadwal(reservasi);
37                        break;
38
39                    case 2:
40                        prosesBoking(scanner, reservasi);
41                        break;
42
43                    case 3:
44                        running = false;
45                        break;
46
47                    default:
48                        System.out.println("[INFO] Pilihan tidak tersedia. Pilih 1-3.");
49                }
50            }
51        }
52    }
53 }

```

Gambar 1. Source code App.java

- b. Kereta.java: Menyimpan data model kereta api (kode, nama, rute, sisa kursi) dan method helper untuk manipulasi kapasitas.

```

1 public class Kereta {
2     private String kode;
3     private String nama;
4     private String rute;
5     private int sisaKursi;
6
7     public Kereta(String kode, String nama, String rute, int kapasitas) {
8         this.kode = kode;
9         this.nama = nama;
10        this.rute = rute;
11        this.sisaKursi = kapasitas;
12    }
13
14    public String getKode() { return kode; }
15    public String getNama() { return nama; }
16    public String getRute() { return rute; }
17    public int getSisaKursi() { return sisaKursi; }
18
19    public void kurangiKursi(int jumlah) {
20        this.sisaKursi -= jumlah;
21    }
22
23    @Override
24    public String toString() {
25        return String.format(kode + " | " + nama + " | " + rute + " | " + sisaKursi);
26    }
27 }
28

```

Gambar 2. Source code Kereta.java

- c. Reservasi.java: Kelas bisnis yang berisi daftar data kereta dan logika transaksi pemesanan tiket dengan penanganan validasi input penumpang dan ketersediaan kursi.

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Reservasi {
5     private List<Kereta> daftarKereta;
6
7     public Reservasi() {
8         daftarKereta = new ArrayList<>();
9         daftarKereta.add(new Kereta(kode: "K01", nama: "Argo Bromo", rute: "JKT - SBY", kapasitas: 50));
10        daftarKereta.add(new Kereta(kode: "K02", nama: "Parahyangan", rute: "JKT - BDG", kapasitas: 15));
11    }
12
13    public List<Kereta> getDaftarKereta() {
14        return daftarKereta;
15    }
16
17    public void pesanTiket(String kodeKereta, String nik, String namaPenumpang, int jumlahTiket)
18        throws RuteTidakDitemukanException, TiketHabisException {
19
20        if (nik.length() != 16) {
21            throw new DataPenumpangTidakValidException(
22                "NIK harus 16 karakter. NIK yang dimasukkan: " + nik.length() + " karakter.");
23        }
24        for (char c : nik.toCharArray()) {
25            if (!Character.isDigit(c)) {
26                throw new DataPenumpangTidakValidException(
27                    "pesan: NIK tidak boleh mengandung huruf atau karakter selain angka.");
28            }
29        }
30
31        Kereta kereta = null;
32        for (Kereta k : daftarKereta) {
33            if (k.getKode().equalsIgnoreCase(kodeKereta)) {
34                kereta = k;
35                break;
36            }
37        }
38
39        if (kereta == null) {
40            throw new RuteTidakDitemukanException(
41                "Kode kereta " + kodeKereta + " tidak ditemukan dalam sistem.");
42        }
43
44        if (jumlahTiket > kereta.getSisaKursi()) {
45            throw new TiketHabisException(
46                "pesan: Tiket tidak mencukupi untuk pemesanan ini.",
47                kereta.getNama(),
48                kereta.getSisaKursi());
49        }
50    }
51 }
```

Gambar 3. Source code Reservasi.java

- d. DataPenumpangTidakValidException.java: Kelas custom unchecked exception (extends RuntimeException) untuk menangani kesalahan input NIK (bukan 16 digit angka).

```
1 public class DataPenumpangTidakValidException extends RuntimeException {
2     public DataPenumpangTidakValidException(String pesan) {
3         super(pesan);
4     }
5 }
6
```

Gambar 4. Source code DataPenumpangTidakValidException.java

- e. RuteTidakDitemukanException.java: Kelas custom checked exception (extends Exception) untuk menangani ketika kode kereta yang dimasukkan tidak terdaftar.

```
1 public class RuteTidakDitemukanException extends Exception {
2     public RuteTidakDitemukanException(String pesan) {
3         super(pesan);
4     }
5 }
6
```

Gambar 5. Source code RuteTidakDitemukanException.java

- f. TiketHabisException.java: Kelas custom checked exception (extends Exception) yang membawa metadata nama kereta dan sisa kursi yang tersedia.

```

1 public class TiketHabisException extends Exception {
2     private String namaKereta;
3     private int sisaKursi;
4
5     public TiketHabisException(String pesan, String namaKereta, int sisaKursi) {
6         super(pesan);
7         this.namaKereta = namaKereta;
8         this.sisaKursi = sisaKursi;
9     }
10
11     public String getNamaKereta() {
12         return namaKereta;
13     }
14
15     public int getSisaKursi() {
16         return sisaKursi;
17     }
18 }
19

```

Gambar 6. Source code TiketHabisException.java

2. Analisis Penerapan Exception Handling pada Codebase

1) Try-Catch-Finally Block:

Mekanisme ini digunakan di kelas App.java untuk menangani kesalahan input. Ketika pengguna menginput menu utama, InputMismatchException ditangkap untuk mencegah program berhenti jika diinput karakter non-angka. Pada method prosesBoking, NumberFormatException ditangkap jika input jumlah tiket bukan angka. Blok finally digunakan setelah perulangan menu utama selesai (opsi Keluar) untuk memastikan scanner ditutup dengan aman (resource cleanup) dan menampilkan pesan terima kasih.

2) Checked vs Unchecked Exception:

- DataPenumpangTidakValidException didefinisikan sebagai Unchecked Exception (turunan dari RuntimeException). Hal ini karena kesalahan input NIK (panjang tidak 16 digit atau mengandung huruf) merupakan kesalahan logika pengguna yang seharusnya dapat divalidasi dan dihindari di level aplikasi sebelum pemrosesan lebih lanjut. Unchecked exception tidak wajib dideklarasikan menggunakan kata kunci throws pada method.
- RuteTidakDitemukanException dan TiketHabisException didefinisikan sebagai Checked Exception (turunan dari Exception). Keadaan rute tidak ditemukan atau kursi kereta penuh merupakan kondisi eksternal (business logic constraints) yang di luar kendali langsung kode program saat runtime. Checked exception mewajibkan method yang berpotensi melempar eksepsi tersebut untuk mendeklarasikannya di signature method menggunakan kata kunci throws, dan pemanggil method wajib menanganinya menggunakan blok try-catch atau meneruskannya kembali.

3) Custom Exception dengan Atribut Tambahan:

TiketHabisException dirancang dengan atribut tambahan berupa namaKereta (String) dan sisaKursi (int). Melalui constructor kustom, exception ini membawa data relevan saat dilempar, sehingga block catch di App.java dapat mengakses informasi tersebut menggunakan getter dan menampilkan pesan kesalahan yang sangat detail dan informatif kepada pengguna (misal: menunjukkan nama kereta dan berapa sisa kursi yang sebenarnya tersedia).

4) Exception Propagation (Throw & Throws):

Di kelas Reservasi.java, jika validasi gagal, eksepsi dilempar menggunakan kata kunci throw (misal: throw new TiketHabisException(...)). Deklarasi throws pada method

pesanTiket menandakan bahwa method tersebut menyebarkan checked exception ke pemanggilnya, yaitu method prosesBooking di App.java, di mana eksepsi tersebut akhirnya ditangkap (catch) dan ditangani secara visual.

V. Hasil output Program

Pengujian program dilakukan untuk memverifikasi fungsionalitas dan ketangguhan exception handling dalam berbagai skenario:

Skenario 1 (Lihat Jadwal Kereta): Pengguna memilih menu 1 untuk melihat daftar kereta, rute, dan kapasitas kursi yang tersedia.

```
=====
Selamat Datang di JAVA EXPRESS
Sistem Reservasi Tiket Kereta Api
=====

--- MENU UTAMA ---
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu: 1

=====
JADWAL KERETA TERSEDIA
=====
Kode | Nama Kereta | Rute | Sisa Kursi
-----
K01 | Argo Bromo | JKT - SBY | 50
K02 | Parahyangan | JKT - BDG | 15
=====
```

Gambar 7. Skenario Lihat Jadwal Kereta

Skenario 2 (Pemesanan Tiket Berhasil): Pengguna berhasil memesan tiket dengan menginput data yang valid (Kode K01, NIK 16 digit angka, jumlah tiket 1). Sisa kursi berkurang secara otomatis.

```
--- MENU UTAMA ---
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu: 2

--- PEMESANAN TIKET ---
Kode Kereta : K01
NIK Penumpang : 1234567890123456
Nama Penumpang: zulka
Jumlah Tiket : 1

=====
PEMESANAN BERHASIL!
=====
Kereta : Argo Bromo
Rute : JKT - SBY
Penumpang : zulka
NIK : 1234567890123456
Jumlah Tiket : 1
Sisa Kursi : 49
=====
```

Gambar 8. Skenario Pemesanan Berhasil

Skenario 3 (Error NIK Tidak Valid): Pengguna menginput NIK yang tidak valid (kurang dari 16 digit). Sistem menangkap DataPenumpangTidakValidException dan menampilkan pesan error tanpa menghentikan aplikasi.

```
--- MENU UTAMA ---
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu: 2

--- PEMESANAN TIKET ---
Kode Kereta : k01
NIK Penumpang : 1234
Nama Penumpang: fufa
Jumlah Tiket : 1
[ERROR] Data penumpang tidak valid: NIK harus 16 karakter. NIK yang dimasukkan: 4 karakter.
```

Gambar 9. Skenario Error NIK Tidak Valid

Skenario 4 (Error Rute Tidak Ditemukan): Pengguna menginput kode kereta yang tidak terdaftar (K03). Sistem menangkap `RuteTidakDitemukanException` dan membatalkan transaksi dengan aman.

```
--- MENU UTAMA ---
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu: 2

--- PEMESANAN TIKET ---
Kode Kereta : K03
NIK Penumpang : 1234567890123456
Nama Penumpang: niel
Jumlah Tiket : 1
[ERROR] Rute tidak ditemukan: Kode kereta 'K03' tidak ditemukan dalam sistem.
```

Gambar 10. Skenario Error Rute Tidak Ditemukan

Skenario 5 (Error Tiket Habis / Tidak Mencukupi): Pengguna memesan tiket melebihi sisa kursi (pesan 20 tiket untuk kereta K02 yang hanya sisa 15). Sistem menangkap `TiketHabisException` dan menampilkan informasi sisa kursi yang tersedia.

```
--- MENU UTAMA ---
1. Lihat Jadwal Kereta
2. Pesan Tiket
3. Keluar
Pilih menu: 2

--- PEMESANAN TIKET ---
Kode Kereta : K02
NIK Penumpang : 0987654321123456
Nama Penumpang: shiina
Jumlah Tiket : 20
[ERROR] Tiket tidak mencukupi!
Kereta : Parahyangan
Tersisa : 15 kursi
```

Gambar 11. Skenario Error Tiket Habis

VI. Kesimpulan

Penerapan Exception Handling pada aplikasi Java Express berhasil membuktikan efisiensinya dalam mengantisipasi kesalahan runtime dan menjaga kestabilan program. Penggunaan try-catch-finally mencegah kegagalan fatal (crash) akibat kesalahan input pengguna dan menjamin pelepasan sumber daya (scanner) yang bersih. Pemisahan checked exception (untuk kondisi bisnis luar biasa seperti rute tidak ditemukan dan tiket habis) dan unchecked exception (untuk kesalahan input data penumpang) memberikan struktur penanganan yang logis dan disiplin. Selain itu, custom exception dengan parameter tambahan terbukti efektif dalam mentransmisikan data error dari modul logika bisnis (`Reservasi.java`) ke modul tampilan (`App.java`) guna menyajikan pesan error yang informatif kepada pengguna.

VII. Daftar Pustaka

Oracle. (2026). Exceptions in Java. Retrieved from Oracle Java Documentation: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>

GeeksforGeeks. (2026). Exception Handling in Java. Retrieved from GeeksforGeeks:
<https://www.geeksforgeeks.org/exceptions-in-java/>